NATIONAL TEACHERS COLLEGE

Book Reservation Management School of Arts, Sciences and Technology | Department of Computer Studies CC105 – DATA STRUCTURE W/LAB

TERMINAL ASSESSMENT 2 / MILESTONE 2 STUDENT WORKSHEET

Title: Dynamic Data Engine & Benchmarking Suite

PAGE 1 – STUDENT AND GROUP INFORMATION

Section: 2.4 BSIT

Group Number: _____

Project/Scenario Title: Scenario 3: Smart Library Transaction & Archival System

Date: October 25 , 2007

Repository URL:
https://github.com/425002643-debug/Dynamic-Data-Engine-Benchmarking-Suite-Worksheet.git

GROUP MEMBERS (THREE (3) TO FIVE (5) MEMBERS ONLY)

*Notice: Terminal Assessment 2 must be completed by a group of **THREE (3) to FIVE (5) members**. Every member must participate in development, testing, documentation, demonstration, and practical defense.*

No.	Student Name	Assigned Role	Assigned Module	Signature
1	Serdenia, Sheayla Rihanna A.	Linked List & Catalog Developer	Linked List Module	
2	Cunanan, Jhoey	Stack & Undo/Audit Developer	Stack Module	
3	Mier, Ronamae	Queue & Priority Queue Developer	Queue & Priority Queue Modules	
4	De Pedro, Precious Mae M.	Search & Sort Developer	Searching & Sorting Module	
5	Quintela, Nicole M.	Benchmarking & CLI Developer	Benchmarking & CLI Module	

PAGE 2 – PROJECT PROBLEM AND OBJECTIVES

PART A – PROJECT SCENARIO

Project Title: Library Catalog, Reservation & Return Management Engine

Selected / Assigned Scenario: Scenario 3: Smart Library Transaction & Archival System

Describe the real-world problem: Many School and community libraries still stack books, reserve, borrow, and return using manual logbooks or spreadsheets. As the collection and the number of borrowers grow, searching for books, checking their availability, handling reservation queues, and computing overdue returns become slow and error-prone. Records of past transactions pile up with no structured archive, so old data either clutters the active system or gets lost. A library needs a system that processes transactions quickly, keeps them in order, and moves old records to an archive without slowing daily operations.

Who are the intended users?

- **Librarians:** Who add and update catalog entries, process borrowing and returning, and manage reservations.
- **Students/Faculty/Borrowers:** Who search the catalog, reserve books that are currently borrowed, and check their borrowing status.
- **Library Administrators:** Review transaction history and archived records, and generate reports.

What problem will the application solve?

The application will provide a fast, organized engine for the library's core transactions. It will let users search the catalog efficiently, enforce first-come-first-served reservation queues for borrowed books, record borrow and return transactions with due dates, and automatically move completed or old transactions into an archive. It will also include a benchmarking component that compares the performance of the chosen data structures (for example, hash tables or balanced trees versus linear search, and queues versus lists) as the dataset grows, which shows why those structures were chosen.

PART B – PROJECT OBJECTIVES

General Objective: To design and implement a C++ library transaction engine that uses linked lists, stacks, and queues to manage catalog records, reservations, returns, and to evaluate search and sort algorithms by comparisons, swaps, and execution time.

Specific Objectives:

1. **Catalog (linked list):** Add, delete, search, update, and display book records, with duplicate-ISBN checking and no memory leaks.
2. **Stack:** The returned-book drop stack and undo/audit trail.
3. **Queues:** FIFO hold queue (plus the priority hold queue if you build it).
4. **Search and Sort:** Linear and binary search, plus bubble, selection and insertion sort, with comparison swap counters.
5. **Benchmarking and CLI:** The benchmarking suite with timing and report table, the menu-driven interface.

PAGE 3 – SYSTEM REQUIREMENTS

Identify at least **FIVE (5)** major system functions required for your application:

No.	System Function	Description	Data Structure Needed
1	Master Book Catalog	Stores book information and allows adding, deleting, updating, and displaying records dynamically.	Doubly Linked List
2	Book Return Management	Manages returned books using Last In, First Out(LIFO) processing.	Stack
3	Book Reservation Management	Manages the book reservations according to the order in which the patron requested.	Queue
4	Book Searching	Supports title based linear searching and ISBN based binary searching.	Linked List/Sorted Array
5	Catalog Sorting and Benchmarking	Evaluates searching and sorting algorithms using comparison, counts, swaps and execution time.	Dynamic Array

REQUIRED APPLICATION CHARACTERISTICS CHECKLIST

Verify that your application satisfies all mandatory technical criteria:

Menu-driven C++ terminal application
Modular program structure (multi-file layout or clean function separation)
Robust input validation (prevents crashes on invalid data types)
Exception/Error handling routines
Dynamic memory management (`new`, `delete`, pointers)
Search functionality implementation
Sorting functionality implementation
Benchmarking functionality (swap/comparison counts and execution profiling)
Proper memory cleanup (zero memory leak execution)
Meaningful variable and function naming conventions
In-code documentation and comments
Application compiles successfully with zero warnings/errors

PAGE 4 – DATA STRUCTURE DESIGN WORKSHEET

Plan the integration of dynamic data structures in your software architecture:

System Feature	Data Structure	Why Selected?	Major Operations	Expected Big-O
Master Book Catalog	Doubly Linked List	It supports dynamic records and bidirectional traversal.	Insert, Delete, Search, Update, Display.	O(n) worst case
Book Return Management	Stack	It processes the recently returned books using LIFO.	Push, Pop, Peek	O(1)
Book Reservation Management	FIFO Queue	It processes the reservations based on their arrival order.	Enqueue, Dequeue, Peek.	O(1)
Book Searching	Linked List/Sorted Dynamic Array	It supports sequential title searching and binary ISBN searching.	Linear, Binary Search.	O(n) / O(log n)
Catalog Sorting and Benchmarking	Dynamic Array	It supports indexed access for sorting operations.	Bubble, Selection, Insertion.	$O(n^2)$ worst case

Consider using: Singly Linked List, Doubly Linked List, Circular Linked List, Stack, Queue, Circular Queue, Priority Queue.

DESIGN ANALYSIS QUESTIONS

1. **Which data structure is the MOST important in your application?**

The most important data structure in our application is the Doubly Linked List. Because the Doubly Linked List serves as the primary storage structure of our Master Book Catalog. It allows the book records to be dynamically inserted, deleted, updated, and displayed without requiring a fixed storage capacity. Each node contains book information and pointers that connect to the previous and next nodes.

2. **Which data structure was the most difficult to implement?**

The data structure that was difficult to implement was the Doubly Linked List. It is difficult since it requires careful pointer manipulation because each node contains two pointers: the prev and next. When we insert or delete a book, these pointers must be updated correctly to maintain the connections between nodes. Incorrect pointer reassignment may result in broken connections or memory access errors.

PAGE 5 – LINKED LIST IMPLEMENTATION

Linked List Type Used:

Doubly Linked List

Purpose in the System:

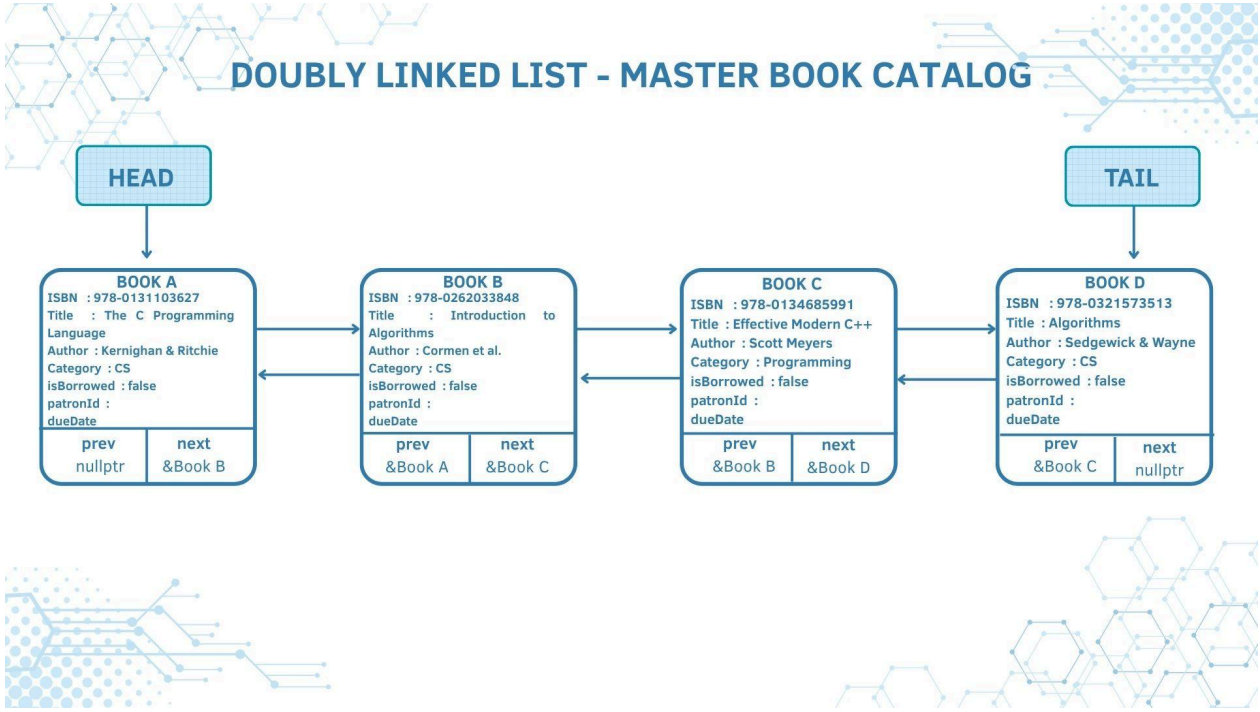
The Doubly Linked List serves as the main storage structure for our Smart Library System's Master Book Catalog. It stores the essential book information such as the ISBN, title, author, category, borrowing status, patron ID, and the due date. Each book is represented by a dynamically allocated node connected to the other nodes through prev and next pointers. This structure allows the system to efficiently manage book records through insertion, deletion, updating, searching, and traversal in both forward and backward directions.

Documented Operations:

Insert / Add
Delete / Remove
Search / Find
Update / Edit
Display / Traverse

Operation	Function Name	Technical Description	Big-O Complexity
Insert	insertHead() / insertTail()	Validates ISBN uniqueness and adds a book at the beginning or end.	O(n)
Delete	deleteByIsbn()	It locates a book, reconnects neighboring nodes, and releases its memory.	O(n)
Search	findByIsbn()	Traverses the list to locate a matching ISBN.	O(n)
Update	updateBook()	Updates an existing book record without changing node connection.	O(n)
Traverse	displayForward() / displayBackward()	Displays records from head to tail or vice versa.	O(n)

NODE / POINTER STRUCTURAL DIAGRAM



PAGE 6 – STACK IMPLEMENTATION

Purpose of Stack in System:

The stack serves as an audit trail and undo or rollback mechanism for the library management system. It records state modifications such as book insertions, status updates, and record deletions in a strict Last-In, First-Out (LIFO) order.

Underlying Implementation:

Dynamic Array
Linked List
Other: STD::STACK adaptor back by std::degue

Documented Operations:

Push [x]
Pop [x]
Peek / Top [x]
Display / Traverse [x]

Operation	Function Name	Big-O Complexity	Verified & Tested?
Push	AuditStackModule::push	O(1)	[X]
Pop	AuditStackModule::pop	O(1)	[X]
Peek	AuditStackModule::peek	O(1)	[X]

Operation	Function Name	Big-O Complexity	Verified & Tested?
Display	auditstackmodule::display	O(N)	[X]

Explain how the LIFO (Last-In, First-Out) principle is utilized in your application:

The library has the LIFO principle and is utilized through the auditstackmodule, which acts as an audit log and undo system. Under the LIFO structure, the most recent action performed is always placed on top of the stack and is the first to be removed.

PAGE 7 – QUEUE / PRIORITY QUEUE IMPLEMENTATION

Queue Type Used:

Standard Queue
Circular Queue
Priority Queue

Purpose of Queue in System:

To manage the library book reservations fairly and efficiently. The **Standard Queue** follows a first-come, first-served policy so they can get their books in the order they requested. The **Priority Queue** lets urgent or high-priority reservations jump ahead of regular requests based on their assigned tier.

Documented Operations:

Enqueue
Dequeue
Front / Peek
Display / Traverse

Operation	Function Name	Big-O Complexity	Verified & Tested?
Enqueue	enqueueFIFO / enqueuePriority	O(1) FIFO/ O(n) Priority	[x]
Dequeue	dequeueFIFO / dequeuePriority	O(1)	[x]
Peek	peekFIFO / peekPriority	O(1)	[x]
Display	displayFIFOstate / displayPriorityState	O(n)	[x]

Explain why FIFO (First-In, First-Out) or Priority-based processing is appropriate:

FIFO is appropriate for a standard library hold request because it ensures fairness, treating all regular borrowers equally by serving them in the exact order their reservations were submitted.
Priority-based processing is appropriate when certain reservations require special urgency or priority (such as rush academic materials or faculty requests) so they can bypass standard lines based on their tier.

PAGE 8 – SEARCHING ALGORITHMS

Implemented Search Algorithms:

Linear Search
Binary Search

SEARCH ALGORITHM COMPARISON TABLE

Criteria	Linear Search	Binary Search
Best Case Complexity	O(1)	O(1)
Average Case Complexity	O(n)	O(log n)
Worst Case Complexity	O(n)	O(log n)
Space Complexity	O(1)	O(1)
Requires Sorted Data?	No	Yes
System Function Name	linearSearch	binarySearch

SEARCH EVALUATION QUESTIONS

1. **Which search algorithm performed better during empirical testing?**

Answer: Binary Search operates in logarithmic time (O(\log n)) by repeatedly dividing the search interval in half. For larger datasets, it drastically reduces the number of comparisons required compared to Linear Search, which must check elements one by one in linear time (O(n)).

2. **Why must the dataset normally be sorted before applying Binary Search?**

Answer: Binary Search relies on the property of monotonicity (ordered data) to determine whether the target value lies in the left or right half of the current range. If the data is unsorted, eliminating half of the elements based on a mid-point comparison would be completely invalid, as smaller or larger elements could appear anywhere.

PAGE 9 – SORTING ALGORITHMS

Implemented Sorting Algorithms:

Bubble Sort
Selection Sort
Insertion Sort

SORTING ALGORITHM METRICS RECORD

Algorithm	Function Name	Dataset Size (N)	Comparisons	Swaps	Execution Time (ms)	Theoretical Big-O
Bubble Sort	Bubble Sort	100	4,814	2,387	0.029ms	$O(n^2)$
Bubble Sort	Bubble Sort	1,000	499,200	253,155	2.266ms	$O(n^2)$
Selection Sort	Selection Sort	100	4,950	95	0.015ms	$O(n^2)$

Algorithm	Function Name	Dataset Size (N)	Comparisons	Swaps	Execution Time (ms)	Theoretical Big-O
Selection Sort	Selection Sort	1,000	499,500	991	1.058ms	$O(n^2)$
Insertion Sort	Insertion Sort	100	2,481	2,387	0.009ms	$O(n^2)$
Insertion Sort	Insertion Sort	1,000	254,145	253,155	0.761ms	$O(n^2)$

Fastest Algorithm Observed:
Insertion Sort (especially on nearly sorted or average random data due to fewer total comparisons and early exits/inner-loop efficiency).

Slowest Algorithm Observed:
Bubble Sort (due to a high frequency of unnecessary swaps and comparisons).

Explain the observed metric results:

All three algorithms share a quadratic theoretical time complexity (O(n^2)), meaning their execution times and comparison counts skyrocket quadratically when the dataset size (N) increases from 100 to 1,000.

Insertion Sort generally outperforms the other two in practice because it performs half as many comparisons on average as Bubble/Selection Sort for random data and can break early.

Selection Sort consistently yields the lowest number of swaps (roughly N swaps max), but its comparison count remains identical to Bubble Sort because it always scans the entire remaining sublist.

PAGE 10 – BENCHMARKING WORKSHEET

Enter the empirical execution results recorded by your benchmarking module. Do not use theoretical guesses.

BENCHMARK RESULTS TABLE

Algorithm	Dataset Size	Comparisons	Swaps	Execution Time	Theoretical Big-O
Bubble Sort	Small (N=10)	35	19	0.001	O(n²)
Bubble Sort	Medium (N=500)	124120	65743	0.849	O(n²)
Bubble Sort	Large (N=5000)	12491395	6251385	74.611	O(n²)

Algorithm	Dataset Size	Comparisons	Swaps	Execution Time	Theoretical Big-O
Selection Sort	Small (N=10)	45	7	0.001	O(n²)
Selection Sort	Medium (N=500)	124750	495	0.368	O(n²)
Selection Sort	Large (N=5000)	12497500	4993	28.684	O(n²)

EMPIRICAL ANALYSIS QUESTIONS

1. **Did execution time scale as expected when dataset size increased?**

Explain: Mostly yes. Going from N=500 to N=5000 multiplies N by 10, so an O(n²) sort should do about 100 times more work. Bubble Sort's comparisons grew from 124,120 to 12,491,395 (about 100×), and Selection Sort's from 124,750 to 12,497,500 (about 100×). Execution time grew by a similar amount. Bubble Sort rose from 0.849 ms to 74.611 ms (about 88×) and Selection Sort from 0.368 ms to 28.684 ms (about 78×). Both are close to the predicted 100×, but a bit under it. At N=500, the run takes under a millisecond, so fixed overhead and measurement noise make up a larger share of the time. At N=10, the 0.001 ms times are at the limit of the timer's resolution, so they can't be compared meaningfully.

---

2. **Did your empirical results match theoretical Big-O predictions?**

Explain: Yes. Both sorts are $O(n^2)$, and the comparison counts follow it closely. Selection Sort's comparisons exactly equal $n(n-1)/2$: 45 at N=10, 124,750 at N=500 and 12,497,500 at N=5000. Bubble Sort came in slightly lower (124,120 and 12,491,395) because it can stop early once a pass has no swaps. The swap counts show a difference that Big-O doesn't capture. Bubble Sort made 6,251,385 swaps at N=5000, while Selection Sort made only 4,993, which grows linearly. Both algorithms are still $O(n^2)$ because their comparison counts dominate, but swap cost explains why Selection Sort was about 2.6 times faster at N=5000 (28.684 ms against 74.611 ms).

---

3. **What hardware or execution factors cause actual execution time to differ from theoretical Big-O?**

Actual execution time differs from theoretical Big-O because of hardware and execution factors such as CPU cache and memory access, branch prediction, compiler optimizations, operating system activity, and changes in CPU frequency or temperature. Timer resolution and measurement noise can also affect results, especially for very short runs. Additionally, algorithms with the same Big-O complexity may perform differently because of constant factors, such as the number of comparisons and swaps they perform.

PAGE 11 – DYNAMIC MEMORY MANAGEMENT

Dynamic Memory Features Implemented:

Explicit Pointers (`*`, `->`)
Dynamic Allocation (`new`)
Dynamic Deallocation (`delete`/`delete[]`)
Dynamic Node Management
Destructors / Explicit Memory Cleanup Routines

MEMORY MANAGEMENT AUDIT TABLE

Code Component	Memory Allocated?	Memory Released?	Method / Function Used	Verified (No Leak)?
Linked List Nodes	yes	yes	New node() / delete noteptr	[x]
Stack Memory	yes	yes	Std::stack / or delete[] for dynamic stack array	[x]
Queue Memory	yes	yes	Std::queue / or delete[] for dynamic stack array	[x]
Dynamic Arrays	yes	yes	New int[capacity] / delete [] data	[x]
System Destructors	N/A	yes	Class Destructor / classname()	[x]

MEMORY AUDIT QUESTIONS

1. **What is a memory leak, and why is it dangerous in high-performance applications?**

A memory leak occurs when a program dynamically allocates heap memory but fails to release or deallocate that memory, either because it's no longer needed or because references to it are lost

2. **How specifically does your codebase prevent memory leaks during runtime and termination?**

We use C++ built-in STD::stack, which handles memory automatically whenever we push or pop logs. When ending, as soon as the stack is done or the program finishes, C++ will automatically clear and delete all the leftover data from the stack memory for us

PAGE 12 – TESTING WORKSHEET

TEST-CASE EXECUTION LOG

Test #	Target Module	Input Data	Expected Output	Actual Output	Pass/Fail
1. Input Processing	Linked List Module	ISBN: 100 Title: The Art of Programming Author: John Smith Category: Technology	Book added	“Book Added”	Pass
2. Invalid Input Rejection	Linked List Module	ISBN: Title: The Mystery of the Lost Code Author: Alex Carter Category: Fiction	Insert Failed	“Insert Failed” (empty duplicate ISBN, or insert not implemented yet)	Pass
3. Empty structure operation	Queue Modules	Menu 2 (Hold Reservation Queues) Option 3 (Dequeue FIFO)	Queue is Empty	“Queue is Empty”	Pass
4. Single Record Operation	Linked List Module	ISBN: 1001 Title: Programming Fundamentals Author: John Meyer Category: Programming	The one book is shown	One book record was successfully inserted and displayed.	Pass
5. Multiple Record Operation	Linked List Module	ISBN:1001, 1002, 1003	Forward and backward show the same book in opposite order	All three book records were successfully displayed.	Pass
6. Existing Record Search	Linked List Module	ISBN: 1002	The row for that book appears	Book found: DBMS.	Pass
7. Non-Existing Record Search	Linked List Module	ISBN: 999	Book not found	Not found	Pass
8. Node deletion handling	Linked List Module	ISBN to delete: 1002	Each is removed, and the display	Book deleted. ISBNs 1001 and 1003 remained.	Pass

			shows no gaps.		
9. Sorting functionality verification	Sorting Module	Benchmark run at N = 10	The sorted part shows a column in all as yes for all algorithms	“Benchmark Complete” The Sorted? All “Yes”	Pass
10. Stack/Queue boundary conditions	Stack & Queue Modules	Menu 3 (Audit Trail / Undo) Option 1, 2, 3 Menu 2 (Hold Reservation Queues) Option 3, 1, 3, 3	Nothing to undo and the Queue is empty	Shows “Audit Trail is Empty” “Nothing to Undo” Shows “Queue is Empty”	Pass

MANDATORY TESTING COVERAGE CHECKLIST

Valid input processing
Invalid input rejection
Empty structure operations (underflow)
Single record operations
Multiple record operations
Existing record search
Non-existing record search
Node deletion handling
Sorting functionality verification
Stack/Queue boundary conditions

PAGE 13 – BUG AND DEBUGGING LOG

Record major bugs encountered during system development and debugging:

No.	Error / Bug Encountered	Root Cause	Solution Applied	Result
1	Compiler error: “redefinition of struct Book”	Book was defined in both EntityModel.h and AuditStackModule.h, and main.cpp includes both.	Deleted the duplicate struct from the stack header and added #include "EntityModel.h".	Error gone. Project uses one shared Book.
2	Errors: “redefinition of ActionType, AuditLog, AuditStackModule ” and push	The stack .cpp had a full copy of the code pasted at the top, including its own structs and a test main(), on top of the header.	Replaced the .cpp with a clean version holding only the member-function implementations.	Build errors cleared.
3	Stack files not found or duplicated in the project	Header and source names didn’t match the #include lines (StackEngine vs AuditStackModule), and two file pairs were in the project.	Kept one file pair and made every #include use that exact name.	Include errors resolved.
4	Undo of a book insertion couldn’t find which book to remove.	The stack’s pop() only returns the previous state, which is empty for an insertion, so the ISBN was lost.	Stored a placeholder book holding only the ISBN when an insertion is logged.	Undo of an insertion can now identify the right book.
5	“Not declared” error in SearchSortEngine. cpp	Missing #include	Added the missing #include at the top of the SearchSortEngine File.	Error cleared and the file compiled.

DEBUGGING REFLECTION

Most difficult bug encountered during development: The “redefinition of struct Book” error in the audit stack module. It was hard because it caused a long chain of other errors at once (ActionType, AuditLog, AuditStackModule, and the push/pop functions were all reported as redefined). The real problem was small, but the messages made it look like many separate bugs.

How was it identified and resolved?

- When I ran the code, it showed repeated errors related to the Stack and Entity files. After checking the Build messages, I found that the Book struct was defined in both EntityModel.h and the stack header, which caused a conflict. The stack.cpp file also contained duplicate code. I fixed this by removing the duplicate definition, including EntityModel.h in the stack header, and cleaning up the .cpp file. I also checked that the file names and #include statements matched. After rebuilding the project, the errors were gone. From this, I learned to check the first compiler error and make sure the same structure is not defined multiple times.

PAGE 14 – INDIVIDUAL CONTRIBUTION MATRIX

Every group member (3 to 5 members) must have clear technical responsibilities and contribute to git repositories.

Member Name	Assigned Role	Modules Developed	Testing & Documentation	Contribution 100 %
Serdenia, Sheayla Rihanna A.	Linked List & Catalog Developer	Linked List Module	**Page 3 - 5, 12**	22 %
Cunanan, Jhoey	Stack & Undo/Audit Developer	Stack Module	**Page 6, 11**	19 %
Mier, Ronamae	Queue & Priority Queue Developer	Queue & Priority Queue Modules	**Page 7, 19**	19 %
De Pedro, Precious Mae M.	Search & Sort Developer	Searching & Sorting Module	**Page 8 - 9**	17 %
Quintela, Nicole M.	Benchmarking & CLI Developer	Benchmarking & CLI Module	**Page 2, 10, 12 , 13**	23 %

MANDATORY CONTRIBUTION RULES

- Terminal Assessment 2 requires **THREE (3) to FIVE (5) members** per group.
- Every member must make technical commits verified in GitHub history.
- Total contribution percentages must sum up to exactly **100%**.
- Individual defense scores may be adjusted if contribution is unbalanced.
- Every member must be capable of explaining and modifying any portion of the codebase during oral defense.

PAGE 15 – GITHUB AND DELIVERABLE CHECKLIST

PROJECT DELIVERABLES CHECKLIST

Verify all required physical and digital submission assets:

Complete C++ Source Code (`.cpp`, `.h` files)

Active GitHub Repository

Professional `README.md` file in repository

System Technical Documentation
Software Architecture Diagram

Program Flowchart / Data Flow Diagram

System Application Screenshots

Benchmark Results Log & Charts

Big-O Complexity Analysis Section

Completed Test-Case Results Log

Bug & Debugging Log

Completed Individual Contribution Matrix

Presentation Slides / Demonstration Materials

Compiled Executable (where applicable)

GitHub Repository URL:

https://github.com/425002643-debug/Dynamic-Data-Engine-Benchmarking-Suite-Worksheet.git

Latest Commit Hash:

1e29b375255f3a0b468d8378b1701dd4dab2166d

PAGE 16 – LIVE DEMONSTRATION CHECKLIST

For Evaluator Use During Practical Defense:

LIVE DEMONSTRATION EVALUATION ITEMS

- Program compiles successfully without warnings/errors
- Main Menu interface functions smoothly
- Linked List CRUD operations function correctly
- Stack push/pop operations function correctly
- Queue / Priority Queue operations function correctly
- Linear Search executes and displays results correctly
- Binary Search executes and displays results correctly
- Bubble Sort executes and sorts accurately
- Selection Sort executes and sorts accurately
- Insertion Sort executes and sorts accurately
- Benchmarking suite accurately counts comparisons/swaps
- Dynamic memory allocation and deallocation verified clean
- Input validation prevents system crash on bad input

EVALUATOR NOTES & OBSERVED RUNTIME ISSUES

PAGE 17 – RANDOM LIVE CODE MODIFICATION

INSTRUCTOR EVALUATION SECTION: The instructor will assign a random live code modification task to test individual code mastery.

Selected Student Name:

_____

Random Modification Assigned:

Modification Examples:

- *Add a new data field to dynamic nodes*
- *Change search criteria from ID to Name*
- *Modify sorting order from Ascending to Descending*
- *Add element counting to Linked List traversal*
- *Implement input threshold bounds checking*
- *Fix intentionally introduced pointer mismatch*

Student Completion Performance:

Completed independently within time limit
Completed with minor guidance
Partially completed
Unable to complete code modification

Instructor Comments:

PAGE 18 – ORAL DEFENSE WORKSHEET

INDIVIDUAL EVALUATION RECORDS

- **Member 1 Name:** Cunanan, Jhoey
  Question Asked: _____
  Score (1-15): _____
- **Member 2 Name:** De Pedro, Precious Mae M.
  Question Asked: _____
  Score (1-15): _____
- **Member 3 Name:** Mier, Ronamae
  Question Asked: _____
  Score (1-15): _____
- **Member 4 Name:** Quintela, Nicole M.
  Question Asked: _____
  Score (1-15): _____
- **Member 5 Name:** Serdenia, Sheayla Rihanna A.
  Question Asked: _____
  Score (1-15): _____

ORAL DEFENSE QUESTION BANK

1. *How do dynamic pointers differ from standard static variables in memory allocation?*
2. *What is the advantage of a Doubly Linked List over a Singly Linked List?*
3. *How does your application handle stack overflow or queue underflow states?*
4. *Why is Binary Search $O(\log n)$ while Linear Search is $O(n)$?*
5. *Explain the step-by-step logic of Insertion Sort during a swap pass.*
6. *What occurs in memory if `delete` is not called on dynamically created nodes?*
7. *How did you count operations (swaps/comparisons) inside your sorting loops?*
8. *What is the difference between time complexity and space complexity?*
9. *Why does a priority queue require modified enqueue insertion logic?*
10. *How is memory laid out for a dynamic node pointer struct/class?*
11. *What happens to pointer links when a middle node is deleted in a list?*
12. *How do empirical execution times correlate with theoretical Big-O curves?*
13. *What input validation mechanisms were placed on user menu input?*
14. *Trace how your program executes a search request from menu option to memory return.*
15. *How did your team resolve git merge conflicts during development?*

PAGE 19 – GROUP REFLECTION

1. **What was the most challenging technical part of the project?**

- The most challenging part was managing dynamic memory correctly and maintaining pointer links. Especially during node deletion in the doubly linked list and handling Queue/stack cleanup to prevent memory leaks.

2. **Which implemented data structure proved most useful for your scenario, and why?**

- The Doubly Linked List was the most useful because it served as the core storage for the book catalog, allowing dynamic addition, deletions, updates, and bidirectional traversal without a fixed size limit.

3. **Which sorting algorithm performed best on large datasets in your benchmark?**

- Selection sort performed better in execution time on large datasets during our benchmark because it makes far fewer swaps than Bubble sort, despite both having an $O(n^2)$ complexity.

4. **What major key takeaway did the group learn regarding Big-O algorithmic complexity?**

- We learned that while theoretical Big-O helps predict scaling behavior as data grows, actual execution times are heavily impacted by the constant factors like the number of swaps, memory access, and hardware performance.

5. **If given additional development time, what system architectural improvements would you implement?**

- If I had more time, I would first improve how the modules fit together, since most of the problems we hit came from the interfaces between them, not from the algorithms themselves. Saving data and more reliable testing. Right now, everything in the system disappears when the program closes. I would make it save the book catalog and the history of changes to a file, so the information is still there the next time it opens. I would also run each speed test several times and use the middle result, because the timings change slightly each time we run them. That makes the results more trustworthy.

PAGE 20 – TERMINAL ASSESSMENT 2 RUBRIC

100-POINT ANALYTIC EVALUATION RUBRIC

Evaluation Criteria	Exemplary	Proficient	Developing	Beginning	Score
Data Structure Implementation *(25 pts)*	Flawless pointer & memory management ; dynamic structures implemented perfectly; zero memory leaks. *(22-25 pts)*	Correct implementati on with minor code inefficiencies or non critical minor leaks. *(18-21 pts)*	Basic implementation present; contains noticeable pointer operational defects. *(14- 17 pts)*	Incomplete, non functional data structure implementati on. *(0-13 pts)*	/ 25
Algorithmic Efficiency, Searching, Sorting & Big-O *(20 pts)*	Exceptional mastery of searching/sorting algorithms & accurate Big O profiling suite. *(18-20 pts)*	Algorithmic logic correct; minor suboptimal routines in benchmark tracking. *(14- 17 pts)*	Suboptimal choices leading to performance degradation. *(10-13 pts)*	Incorrect algorithm selection; execution fails or loops infinitely. *(0-9 pts)*	/ 20
System Integration & Functionality *(20 pts)*	Seamless integration of modules into a menu-driven system; handles all edge cases. *(18-20 pts)*	System integrated with minor bugs that do not crash application. *(14-17 pts)*	Modules operate in isolation with unstable data flow between options. *(10- 13 pts)*	System fails to execute core user features or compile. *(0-9 pts)*	/ 20

Evaluation Criteria	Exemplary	Proficient	Developing	Beginning	Score
Code Quality, Dynamic Memory & Documentation *(20 pts)*	Clean, highly modular C++; robust comments, encapsulation, clean destructors. *(18-20 pts)*	Readable code; standard naming conventions; minor formatting defects. *(14- 17 pts)*	Monolithic or disorganized code structure; missing documentati on. *(10-13 pts)*	Unreadable code failing basic programming standards. *(0-9 pts)*	/ 20
Live Demo, Practical Defense & Mastery *(15 pts)*	Excellent demonstration, flawless live code modification, total team mastery. *(14- 15 pts)*	Smooth demo; successfully completes live code modification with minor guidance. *(11-13 pts)*	Partial completion of live modification; weak answer to defense questions. *(8-10 pts)*	Unable to complete live modification or defend individual code output. *(0-7 pts)*	/ 15

FINAL ASSESSMENT RECORD

Group Base Score: _____ / 100
Individual Adjustment (if applicable): _____
Final Score: _____ / 100

Instructor Comments:

Instructor Signature: _____

Date Evaluated: _____

SCREENSHOTS (EVIDENCE RESULTS)

```
========================================
 Library Catalog, Reservation & Return
 Management Engine
========================================
1. Master Catalog
2. Hold / Reservation Queues
3. Audit Trail / Undo
4. Search Demo (Linear vs Binary)
5. Benchmarking Suite
0. Exit
Choice: 1

--- Master Catalog ---
1. Add book at head
2. Add book at tail
3. Delete book by ISBN
4. Borrow book
5. Return book
6. Find book by ISBN
7. Display forward
8. Display backward
0. Back
Choice:
```

```
--- Hold / Reservation Queues ---
1. Enqueue hold (FIFO)
2. Enqueue hold (Priority)
3. Dequeue FIFO
4. Dequeue Priority
5. Peek FIFO front
6. Peek Priority front
7. Display both queues
0. Back
Choice:
```

```
--- Audit Trail / Undo ---
1. Peek most recent entry
2. Display full trail (newest first)
3. Undo last operation
0. Back
Choice:
```

```
Array size (default 100): 10
Array holds even numbers 2..20.
Target value: 2
Algorithm        Index        Comparisons
Linear           0            1
Binary           0            5
(index -1 means not found)
```

```
--- Benchmarking Suite ---
1. Quick run   (1,000 / 5,000 / 10,000 items)
2. Full run    (1,000 up to 50,000 items - Bubble Sort may take a while)
3. Custom size (single size)
4. Show sort report
5. Show search report
6. Export results to CSV
7. Clear results
8. Show REQUIRED report format (single table, Big-O)
0. Back
Choice:
```

Test Case Result

```
--- Master Catalog ---
1. Add book at head
2. Add book at tail
3. Delete book by ISBN
4. Borrow book
5. Return book
6. Find book by ISBN
7. Display forward
8. Display backward
0. Back
Choice: 1
ISBN: 100
Title: The Art of Programming
Author: John Smith
Category: Technology
Book added.
```

```
--- Hold / Reservation Queues ---
1. Enqueue hold (FIFO)
2. Enqueue hold (Priority)
3. Dequeue FIFO
4. Dequeue Priority
5. Peek FIFO front
6. Peek Priority front
7. Display both queues
0. Back
Choice: 3
Queue is empty.
```

```
--- Master Catalog ---
1. Add book at head
2. Add book at tail
3. Delete book by ISBN
4. Borrow book
5. Return book
6. Find book by ISBN
7. Display forward
8. Display backward
0. Back
Choice: 1
ISBN:
Title: The Mystery of the Lost Code
Author: Alex Carter
Category: Fiction
Insert failed (empty/duplicate ISBN, or insert not implemented yet).
```

```
--- Master Catalog ---
1. Add book at head
2. Add book at tail
3. Delete book by ISBN
4. Borrow book
5. Return book
6. Find book by ISBN
7. Display forward
8. Display backward
0. Back
Choice: 1
ISBN: 100
Title: The Art of Programming
Author: John Smith
Category: Technology
Insert failed (empty/duplicate ISBN, or insert not implemented yet).
```

```
--- Master Catalog ---
1. Add book at head
2. Add book at tail
3. Delete book by ISBN
4. Borrow book
5. Return book
6. Find book by ISBN
7. Display forward
8. Display backward
0. Back
Choice: 1
ISBN: 1001
Title: Programming Fundamentals
Author: John Meyer
Category: Programming
Book added.

--- Master Catalog ---
1. Add book at head
2. Add book at tail
3. Delete book by ISBN
4. Borrow book
5. Return book
6. Find book by ISBN
7. Display forward
8. Display backward
0. Back
Choice: 7

Catalog (1 books) - forward
ISBN            Title                   Author          Category        Status      Patron          Due Date
------------------------------------------------------------------------------------------------------------
1001            Programming Fundamentals  John Meyer          Programming       Available
```

```
--- Master Catalog ---
1. Add book at head
2. Add book at tail
3. Delete book by ISBN
4. Borrow book
5. Return book
6. Find book by ISBN
7. Display forward
8. Display backward
0. Back
Choice: 6
ISBN: 999
Not found.
```

```
--- Master Catalog ---
1. Add book at head
2. Add book at tail
3. Delete book by ISBN
4. Borrow book
5. Return book
6. Find book by ISBN
7. Display forward
8. Display backward
0. Back
Choice: 7

Catalog (3 books) - forward
ISBN            Title                   Author          Category        Status      Patron          Due Date
------------------------------------------------------------------------------------------------------------
1001            Programming Fundamentals  John Meyer          Programming       Available
1002            DBMS                    Alisha Ferer        Database        Available
1003            Computer Networking     Fernando Lee        Networking        Available

--- Master Catalog ---
1. Add book at head
2. Add book at tail
3. Delete book by ISBN
4. Borrow book
5. Return book
6. Find book by ISBN
7. Display forward
8. Display backward
0. Back
Choice: 8

Catalog (3 books) - backward
ISBN            Title                   Author          Category        Status      Patron          Due Date
------------------------------------------------------------------------------------------------------------
1003            Computer Networking     Fernando Lee        Networking        Available
1002            DBMS                    Alisha Ferer        Database        Available
1001            Programming Fundamentals  John Meyer          Programming       Available
```

```
--- Master Catalog ---
1. Add book at head
2. Add book at tail
3. Delete book by ISBN
4. Borrow book
5. Return book
6. Find book by ISBN
7. Display forward
8. Display backward
0. Back
Choice: 6
ISBN: 1002
ISBN            Title                   Author          Category        Status      Patron          Due Date
------------------------------------------------------------------------------------------------------------
1002            DBMS                    Alisha Ferer        Database        Available
```

```
Choice: 3
ISBN to delete: 1002
Book deleted.

--- Master Catalog ---
1. Add book at head
2. Add book at tail
3. Delete book by ISBN
4. Borrow book
5. Return book
6. Find book by ISBN
7. Display forward
8. Display backward
0. Back
Choice: 7

Catalog (2 books) - forward
ISBN            Title                   Author          Category        Status      Patron          Due Date
------------------------------------------------------------------------------------------------------------
1001            Programming Fundamentals  John Meyer          Programming       Available
1003            Computer Networking     Fernando Lee        Networking        Available

--- Master Catalog ---
1. Add book at head
2. Add book at tail
3. Delete book by ISBN
4. Borrow book
5. Return book
6. Find book by ISBN
7. Display forward
8. Display backward
0. Back
Choice: 8

Catalog (2 books) - backward
ISBN            Title                   Author          Category        Status      Patron          Due Date
------------------------------------------------------------------------------------------------------------
1003            Computer Networking     Fernando Lee        Networking        Available
1001            Programming Fundamentals  John Meyer          Programming       Available
```

```
--- Running search benchmarks ---
  Searching in 10 items...

Benchmarks complete.

================================================================================================
SORTING BENCHMARK REPORT
================================================================================================
Algorithm       Pattern             Size     Comparisons    Swaps/Shifts     Time (ms)   Sorted?
------------------------------------------------------------------------------------------------
Bubble Sort     Random                10              35              19         0.002       yes
Selection Sort  Random                10              45               7         0.001       yes
Insertion Sort  Random                10              25              19         0.001       yes
Bubble Sort     Sorted                10               9               0         0.000       yes
Selection Sort  Sorted                10              45               0         0.001       yes
Insertion Sort  Sorted                10               9               0         0.000       yes
Bubble Sort     Reversed              10              45              45         0.001       yes
Selection Sort  Reversed              10              45               5         0.001       yes
Insertion Sort  Reversed              10              45              45         0.001       yes
Bubble Sort     Nearly Sorted         10               9               0         0.000       yes
Selection Sort  Nearly Sorted         10              45               0         0.001       yes
Insertion Sort  Nearly Sorted         10               9               0         0.000       yes
================================================================================================
Note: Insertion Sort 'swaps' counts element shifts, as implemented in SearchSortEngine.
```

```
--- Hold / Reservation Queues ---
1. Enqueue hold (FIFO)
2. Enqueue hold (Priority)
3. Dequeue FIFO
4. Dequeue Priority
5. Peek FIFO front
6. Peek Priority front
7. Display both queues
0. Back
Choice: 1
ISBN: 100
Title: the art of programming
Patron ID: 100
Hold queued with reservation ID 1001.

--- Hold / Reservation Queues ---
1. Enqueue hold (FIFO)
2. Enqueue hold (Priority)
3. Dequeue FIFO
4. Dequeue Priority
5. Peek FIFO front
6. Peek Priority front
7. Display both queues
0. Back
Choice: 3
Processed hold #1001 - the art of programming for patron 100

--- Hold / Reservation Queues ---
1. Enqueue hold (FIFO)
2. Enqueue hold (Priority)
3. Dequeue FIFO
4. Dequeue Priority
5. Peek FIFO front
6. Peek Priority front
7. Display both queues
0. Back
Choice: 3
Queue is empty.
```

Sorting Algorithm Result

```
================================================================================================
                              BENCHMARK SUITE PERFORMANCE METRICS REPORT
================================================================================================
Algorithm       | Dataset Size (N) | Comparisons  | Swaps        | Time Elapsed (ms) | Big-O
----------------+------------------+--------------+--------------+-------------------+----------
Linear Search   | 100              | 51           | N/A          | 0.0003 ms         | O(n)
Binary Search   | 100              | 11           | N/A          | 0.0000 ms         | O(log n)
----------------+------------------+--------------+--------------+-------------------+----------
Bubble Sort     | 100              | 4,814        | 2,387        | 0.029 ms          | O(n^2)
Selection Sort  | 100              | 4,950        | 95           | 0.015 ms          | O(n^2)
Insertion Sort  | 100              | 2,481        | 2,387        | 0.009 ms          | O(n^2)
================================================================================================
```

```
================================================================================================
                              BENCHMARK SUITE PERFORMANCE METRICS REPORT
================================================================================================
Algorithm       | Dataset Size (N) | Comparisons  | Swaps        | Time Elapsed (ms) | Big-O
----------------+------------------+--------------+--------------+-------------------+----------
Linear Search   | 1,000            | 500          | N/A          | 0.0013 ms         | O(n)
Binary Search   | 1,000            | 17           | N/A          | 0.0001 ms         | O(log n)
----------------+------------------+--------------+--------------+-------------------+----------
Bubble Sort     | 1,000            | 499,200      | 253,155      | 2.266 ms          | O(n^2)
Selection Sort  | 1,000            | 499,500      | 991          | 1.058 ms          | O(n^2)
Insertion Sort  | 1,000            | 254,145      | 253,155      | 0.761 ms          | O(n^2)
================================================================================================
```

Benchmark Results

```
================================================================================================
                              BENCHMARK SUITE PERFORMANCE METRICS REPORT
================================================================================================
lgorithm        | Dataset Size (N) | Comparisons  | Swaps        | Time Elapsed (ms) | Big-O
----------------+------------------+--------------+--------------+-------------------+----------
inear Search    | 500              | 251          | N/A          | 0.0008 ms         | O(n)
inary Search    | 500              | 15           | N/A          | 0.0001 ms         | O(log n)
----------------+------------------+--------------+--------------+-------------------+----------
ubble Sort      | 500              | 124,120      | 65,743       | 0.849 ms          | O(n^2)
election Sort   | 500              | 124,750      | 495          | 0.368 ms          | O(n^2)
nsertion Sort   | 500              | 66,239       | 65,743       | 0.259 ms          | O(n^2)
================================================================================================
```

```
================================================================================================
                              BENCHMARK SUITE PERFORMANCE METRICS REPORT
================================================================================================
Algorithm       | Dataset Size (N) | Comparisons  | Swaps        | Time Elapsed (ms) | Big-O
----------------+------------------+--------------+--------------+-------------------+----------
Linear Search   | 5,000            | 2,529        | N/A          | 0.0058 ms         | O(n)
Binary Search   | 5,000            | 22           | N/A          | 0.0001 ms         | O(log n)
----------------+------------------+--------------+--------------+-------------------+----------
Bubble Sort     | 5,000            | 12,491,395   | 6,251,385    | 74.611 ms         | O(n^2)
Selection Sort  | 5,000            | 12,497,500   | 4,993        | 28.684 ms         | O(n^2)
Insertion Sort  | 5,000            | 6,256,380    | 6,251,385    | 20.115 ms         | O(n^2)
================================================================================================
```

```
================================================================================================
                              BENCHMARK SUITE PERFORMANCE METRICS REPORT
================================================================================================
Algorithm       | Dataset Size (N) | Comparisons  | Swaps        | Time Elapsed (ms) | Big-O
----------------+------------------+--------------+--------------+-------------------+----------
Linear Search   | 10               | 6            | N/A          | 0.0001 ms         | O(n)
Binary Search   | 10               | 5            | N/A          | 0.0000 ms         | O(log n)
----------------+------------------+--------------+--------------+-------------------+----------
Bubble Sort     | 10               | 35           | 19           | 0.001 ms          | O(n^2)
Selection Sort  | 10               | 45           | 7            | 0.001 ms          | O(n^2)
Insertion Sort  | 10               | 25           | 19           | 0.001 ms          | O(n^2)
================================================================================================
```